

Expressive Prosody

Syntactic breathing and sculpted F0
for the COVTL articulatory synthesizer

Frédéric Berthommier

fberthommier@users.noreply.github.com

Version 1.1.0 — September 2026

Licensed under GPL-3.0-or-later

Abstract

The expressive prosody option (`expressive=True` / `--expressive`) gives the covtl-pipeline two capabilities that the monotone declination lacks: *syntactic breathing* (short pauses at chunk boundaries, so no chain of coarticulated words exceeds five words) and a *sculpted F0 contour* (pitch accents on stressed syllables, per-chunk attack reset, downstep, terminal nuclear fall). The option is **off by default**: with it off, the pipeline output is bit-identical to the validated monotone chain. The patch acts only on the glottis F0 column and on pause durations — never on the vocal tract — which makes it *speaker-agnostic by construction*: it follows whatever speaker is active in the multi-speaker registry. This guide explains the mechanics, the per-language parameters, how to measure and tune the result, and — in a dedicated chapter — the exact interaction with the speaker setup (`install_speaker.py` / `setup_lang.py` and its JD3 guard).

Contents

1	Why an expressivity option	1
2	Quick start	2
2.1	Command line	2
2.2	Python API	2
3	Syntactic breathing (chunking)	3
3.1	The three passes	3
3.2	Engine marker	3
3.3	Worked example (French)	3
4	The sculpted F0 contour	4
4.1	Where the accents fall	4
4.2	Per-language parameters	4
4.3	Pause tuning	5
5	Word stress: the out-of-band channel	5
6	Measuring and tuning the result	5
6.1	The F0 report	5
6.2	The before/after demo	6
7	Non-regression contract and tests	6

8	Interaction with the speaker setup	7
8.1	The speaker registry in one paragraph	7
8.2	What the expressive patch shares with speakers — exactly one thing	7
8.3	The JD3 guard: scope and behaviour	7
8.4	The two standard workflows	8
8.5	Guarantees matrix	8
9	Limitations (documented heuristics)	9
10	File map	9

1 Why an expressivity option

The monotone chain applies a *linear declination* per phrase (`apply_f0_declination`): F0 starts at *onset gain* and slides down to *final gain*. A long phrase is therefore a flat ramp, perceived as monotonous. Worse, the g2p *connects* all words of a phrase with the syllable separator `.` (coarticulation): a sentence without punctuation becomes a single uninterrupted chain of several seconds, which is hard to parse by ear. Compare the German demo:

```
IC.zu.X@.aj.n@.S2.n@.StRa.s@.In.k2.nICs.bE.ak      <- one 6-word chain
```

Two independent problems, one option:

- **Syntactic breathing** — cut the phrase into breath groups at plausible syntactic boundaries and insert a short pause (~130 ms) between them. No chain exceeds `max_chain_words` (default 5).
- **Sculpted F0** — keep the declination as the base line, and add pitch accents on stressed syllables, an attack reset at every chunk, optional downstep, and a terminal nuclear fall shaped per language.

Design contract (non-negotiable)

1. **Default OFF.** Baselines, scripts and `launch.bat` keep their exact behaviour. With the option off, outputs are bit-identical (enforced by dedicated tests).
2. **Glottis only.** The patch writes the F0 column and the pause durations. It never writes the tract columns, so vowel quality, consonant gestures and the orthogonal branch are untouched.
3. **SAMPA stays pure.** Word stress travels in a parallel, out-of-band channel; the engine key inventory is never polluted.
4. **Documented heuristics.** The chunking is a lexical heuristic, not a parser; the fallback stress rules are orthographic approximations. Both are stated as such here and in the docstrings.

2 Quick start

2.1 Command line

```
# expressive (breathing + sculpted F0); the banner states the mode
vtl-synth run "je cherche une belle route pour aller au travail" \
  --lang fr --expressive -o ./out --no-video

# monotone (the default, stated explicitly)
vtl-synth run "this is easy for us" --lang en --no-expressive -o ./out
```

The SUCCESS banner prints the prosody mode next to the language, mirroring the existing language display:

```
=====
SUCCESS
  langue      : fr (Francais), speaker jd3 -- g2p : text_to_sampa_fr, ...
  prosodie    : expressive (respiration syntaxique + accents de hauteur)
  SAMPA       : b0~ZuR.Z@.SERS % yn.bEl.Rut % puR.a.le.o.tRa.vaj
  ...
=====
```

2.2 Python API

```
from vtl_synth import Pipeline

pipe = Pipeline(lang='fr', expressive=True)
result = pipe.text_to_wav(
    "je cherche une belle route pour aller au travail "
    "parce que le train ne marche plus",
    output_dir="./out", label="demo")
print(result.sampa)
# Z@.SERS.yn.bEl.Rut % puR.a.le.o.tRa.vaj % paRs.k@.l@.tRE~ % n@.maRS.py
```

Requirements and fallbacks:

- Expressive mode requires `use_g2p=True` (phonetic input has no syntactic structure). With `-phonetic` the option is ignored, with a warning, and the transcript keeps no % marker.
- If the module `prosody_f0` is unavailable, or the active language has no `ExpressivityProfile`, the pipeline prints a notice and falls back to the monotone declination.
- The % marker is a regular engine token: you can also hand-write it in SAMPA input (`ba.na.na % ba.da.ga`) even with `-phonetic`; the engine renders the 130ms breath pause, only the F0 sculpting is skipped.

3 Syntactic breathing (chunking)

Module: `v1_synth/utils/chunking.py`.

3.1 The three passes

The chunker runs on the *orthographic* words (before g2p) in three passes:

1. **Lexical boundaries** (rule 2): a boundary is inserted *before* every coordination/subordination conjunction, relative pronoun or connective of the language's function-word list (30–50 items per language). Two-word connectives (*parce que*, *so that*, *bien que*...) are treated as inseparable: the boundary falls *before* the pair, never inside it.
2. **Balanced hard limit** (rule 3): any boundary-free run longer than `max_chain_words` (default 5) is split into near-equal parts — a 7-word run yields 4+3, never 5+2. Balanced cuts also avoid one-word orphans at the end of the sentence.
3. **Dangling function words** (rule 4): a block must not end on a determiner, preposition, auxiliary or clitic when the next block can absorb it without violating the hard limit (*le chien / de mon voisin* → *le chien de / mon voisin* is corrected to *le chien / de mon voisin*).

Already-marked boundaries (commas, semicolons, dashes, sentence punctuation) keep their existing g2p behaviour: comma → 200 ms pause, sentence end → | long pause. The breath marker % sits *between* those two: a dedicated 130 ms pause (`T_pause_breath` in the syltraj engine).

Heuristic, not a parser

The chunking is lexical: no POS tagging, no parse tree. The word lists aim at the 30–50 most frequent function words per language, for roughly 80 % coverage of useful boundaries. Wrong boundaries are possible and benign — a breath at an imperfect spot is far less damaging to intelligibility than a 10-word coarticulated chain.

3.2 Engine marker

In syltraj, % is recognised as a standalone word (like |) and produces a *single* pause node flagged `breath=True` — unlike |, whose surrounding GAP/PAUSE handling was left byte-for-byte untouched. Without any % in the phrase, the parameter `pause_breath_ms` has no effect whatsoever: this is what makes the mode-off behaviour bit-identical.

3.3 Worked example (French)

```
input :  je cherche une belle route pour aller au travail
        parce que le train ne marche plus

chunks:  [je cherche une belle route]
         [pour aller au travail]
         [parce que le train]      <- "parce que" kept together
         [ne marche plus]

SAMPA :  Z@.SERS.yn.bEl.Rut % puR.a.le.o.tRa.vaj % paRs.k@.l@.tRE~ %
        n@.maRS.py
```

4 The sculpted F0 contour

Module: `vtl_synth/utils/prosody_f0.py`.

For every breath group (chunk) of the phrase, the contour is the product of four components:

1. **Base declination, chunk-reset.** The linear onset→final ramp of the language profile is kept, but each chunk recovers part of the attack (`chunk_reset`, 0.6–1.0): the pitch no longer slides down for five seconds without recovering.
2. **Pitch accents.** On the stressed syllable of each accent-bearing word, a smooth asymmetric bump is added: rise ~70 ms, fall 60–90 ms, amplitude `accent_amplitude` as a fraction of `f0_base`. There is no discontinuity — the glottal source follows F0 sample by sample, so smoothness is free; a short 5-frame smoothing still removes numerical corners.
3. **Downstep.** After each accent, the base level is multiplied by `downstep` (< 1 only for German, 0.975 — Ladd 2008); the other languages keep 1.0.
4. **Terminal nuclear fall.** On the last chunk only, the contour is pulled down progressively after the nuclear accent (`nuclear_fall_factor`): deep low terminal for German (0.88) and Portuguese (0.90), contained for French (0.90 on the final accent of the group).

Between chunks, the pause regions get a linear F0 bridge, which guarantees no jump where the voicing of the next attack begins.

4.1 Where the accents fall

- **French:** no lexical accent. One group accent per chunk, on the *last* syllable (Jun & Fougeron 2002: the accent of the accentual phrase is final).
- **Other languages:** accents on the first and last *lexical* word of the chunk (function words are de-accented; a simple given/new heuristic). The in-word position comes from the stress channel (§5).
- If the engine plateaus cannot be aligned with the planned nuclei (fallback), the chunk degrades gracefully to a single group accent on its last syllable.

4.2 Per-language parameters

	amp	rise/fall	reset	downstep	nuclear	reference
en	0.20	70/90 ms	0.7	0.99	0.92	Pierrehumbert 1980; Beckman & Ayers 1997
fr	0.16	70/80 ms	0.6	1.0	0.90	Jun & Fougeron 2002; Di Cristo 1998
es	0.22	70/90 ms	0.8	1.0	0.94	Face 2003; Sosa 1999; Prieto & Roseano 2010
de	0.24	70/60 ms	1.0	0.975	0.88	Féry 1993; Grice et al. 2005 (GToBI)
it	0.22	70/90 ms	0.8	1.0	0.93	Avesani 1995; D’Imperio 2002; Gili Fivela et al. 2015
pt	0.18	70/80 ms	0.7	1.0	0.90	Frota 2000, 2014; Vigário 2003

Why these amplitudes

`amp` is a fraction of `f0_base`; a fixed fraction is a fixed interval in semitones, so 0.16–0.24 corresponds to about 3–4 semitones — the usual range of read-speech pitch accents. A first calibration at 0.10–0.18 (≈ 2 ST) was measured as barely audible and produced a *lower* F0 spread than the monotone ramp; the shipped values were re-tuned from that measurement. Since the fraction multiplies the active speaker’s `f0_base`, the relative relief is identical on every speaker (§8).

4.3 Pause tuning

`pause_breath_ms` (default 130) is the duration of the % pause. It is deliberately shorter than the comma pause (200 ms) and the sentence pause (320 ms). Tune it by ear with `--tpause-breath`—style overrides by editing the `ExpressivityProfile`; 110–160 ms is the useful range.

5 Word stress: the out-of-band channel

The engine never sees the stress: it travels in a parallel dict (`lexicon_loader.stress_for`), mapping word → index of the stressed syllable (vowels emitted). Three sources, in priority order:

1. **English:** the CMUdict stress digits (0/1/2). Primary accent 1 wins over secondary 2. Each ARPAbet vowel is exactly one engine nucleus, so the counting is direct.
2. **Wikipron lexicons** (fr, es, de, it, pt): the marks stress marks (U+02C8 primary / U+02CC secondary) are consumed by `ipa_to_keys_stress` and stored in the channel. *Note:* the broad TSV lexicons currently shipped contain no stress marks (they were stripped upstream), so this source activates only with a narrow-transcription lexicon.
3. **Orthographic fallback rules** (documented approximations):
 - **es:** written accent (áéíóú) → that syllable; otherwise oxytone if the word ends in a consonant other than *n/s*, else penultimate.
 - **it:** written accent; otherwise penultimate.
 - **pt:** written accent; otherwise oxytone on final *-l -r -i -u -im -ns, -ão*, else penultimate.
 - **de:** first syllable of the word (does not model prefixes — documented approximation).

Hiatus handling: a written-accented weak vowel splits the diphthong (*tía* = 2 nuclei, accent on the first). If a word ends up with no stress information at all, the accent falls on its last syllable (group-accent behaviour).

6 Measuring and tuning the result

6.1 The F0 report

```
python scripts/f0_report.py out/demo.tract out2/demo.tract
```

```
fr_long_mono.tract    voiced= 67.4%   dyn= 0.95 ST   span= 25.6 Hz
                      maxjump=25.55 Hz/fr   peaks>+5%= 0
fr_long_expr.tract    voiced= 62.3%   dyn= 0.66 ST   span= 24.0 Hz
                      maxjump= 3.97 Hz/fr   peaks>+5%= 4
```

peaks> +5 % number of *sustained* bumps (at least 30 ms above +5 % of the 400 ms local base).

The expressive files show 3–6; the monotone files show 0. This is the accent counter.

maxjump largest F0 step between consecutive *voiced* frames @400 Hz. The expressive contours stay below ≈ 5 Hz/frame; the monotone declination jumps up to 25–30 Hz/frame at voicing edges of pauses (a pre-existing artefact, untouched by this patch).

dyn standard deviation of the voiced F0 in semitones. Do *not* use it alone as an expressivity score: the monotone ramp maximises it by crawling for seconds at the bottom of its range. Peaks and span are the informative metrics.

6.2 The before/after demo

```
python examples/demo_prosody.py          # the six languages
python examples/demo_prosody.py --summary-only
```

For each language it synthesises a demo pair and a long punctuation-free sentence, both monotone and expressive (24 WAV files in `out_demo_prosody/`), and prints the chain counts and the F0 table. Measured on the reference install (JD3, September 2026):

- chains longer than 5 words: 1–2 per long sentence in monotone, **0 in every expressive file** (12/12);
- sustained pitch accents: 0 monotone everywhere, 3–6 expressive everywhere;
- total duration: expressive adds 4–9% (the breaths).

7 Non-regression contract and tests

```
python -m pytest tests -m "not slow"      # full fast suite
```

`tests/test_expressive.py` (40 tests) covers:

- **bit-identity**: without the % marker, `pause_breath_ms` changes nothing; the monotone transcript is exactly the language g2p output;
- **chunking**: boundary before conjunctions in the six languages, inseparable bigrams, balanced hard limit, dangling function words, comma parts;
- **stress channel**: stress-mark capture (U+02C8/U+02CC), CMUdict digits, orthographic rules per language;
- **F0** (real synthesis): accents measurable ($> +10\%$ above base), smoothness (< 5 Hz/frame between voiced frames), nuclear fall at the end of the last chunk, graceful fallback on plan/segment misalignment;
- **pipeline**: expressive transcripts carry %; phonetic input ignores the mode with a warning.

Baselines per language state

The regression baselines (`regression_baselines*.json`) were captured with the *native/en* vowel targets. With the `fr` LANG SECTION active, the 6 baseline checks fail *by design* — switch to `en` (`python setup_lang.py setlang en`) to get a fully green suite (112/112 on the reference install). This is the same invariant as the language pack's.

8 Interaction with the speaker setup

This chapter describes how the expressive option interacts with the multi-speaker registry and the language pack of the integrated repository. It explains exactly what the expressive patch shares with the speaker machinery, what it does not touch, and what the JD3 guard does and does not restrict.

8.1 The speaker registry in one paragraph

`install_speaker.py` maintains `vtl_synth/data/speakers/`: one registry (`registry.json`), an `ACTIVE_SPEAKER` marker, and per-speaker bundles — a `.speaker` file (geometry and glottis shapes for the orthogonal branch and the VTL wheel) and a `<name>_constants.py` file (the whole of `core/constants.py`, including `f0_default`). Activating a speaker swaps `constants.py` as a whole and swaps the wheel’s speaker resource (effective in the next process); `speaker_registry.announce()` states the active bundle at every synthesis. The language pack (`setup_lang.py`) is independent: it edits the managed LANG SECTION (vowel targets + effort gains) *inside* the installed `constants.py`, whatever speaker it belongs to.

8.2 What the expressive patch shares with speakers — exactly one thing

The sculpted contour reads `self.f0_hz`, which the Pipeline resolves from the *active speaker’s* `f0_default` when not overridden. That is the whole coupling. Everything else the patch does (chunking, % pauses, bump shapes, downstep, nuclear fall) is speaker-independent:

- amplitudes are *fractions* of `f0_base`: a fixed fraction is a fixed musical interval, so the relief is identical in semitones on a 102 Hz male voice (jd3) and on a 181 Hz female voice (s2);
- the patch writes only the glottis F0 column and pause durations — it never writes tract columns, so nothing in the speaker-specific geometry is touched;
- the % pause and `get_last_build()` alignment live in `syltraj`, upstream of any speaker resolution.

Validated on the reference install: expressive synthesis with jd3 (102.216 Hz) and with s2 (181.008 Hz) both produce the expected breath groups and follow the speaker’s F0.

8.3 The JD3 guard: scope and behaviour

`setup_lang.py` carries a guard, `_ensure_jd3()`, invoked at four sites:

command	site	why
<code>install</code>	before writing the LANG SECTION	vowel targets are JD3-calibrated
<code>setlang</code>	before rewriting the section	same, any target language
<code>calibrate</code>	before deriving ρ	in-situ calibration is JD3-only
<code>verify</code>	before the LPC measurement	measurement is JD3-only

Behaviour: if the active speaker is not jd3, the guard *automatically reinstalls jd3* (constants + marker + wheel resource) through `install_speaker.py`, then performs the requested language operation. It is a transport mechanism, not a refusal. Its rationale is narrow: the per-language vowel targets (ρ, θ) of the language pack were calibrated in situ on JD3, so installing or verifying them on another speaker’s constants would silently bake JD3 vowels into that speaker.

What the guard does NOT restrict

- It does not block *any* synthesis. Expressive or monotone, any language, any speaker: the synthesis path (Pipeline / CLI) has no speaker gate.
- It does not prevent *switching speakers*. `install_speaker.py install s2` works at any time — and the speaker swap *carries the active LANG SECTION* over to the new speaker's constants.
- It fires on *every* language-management command, including `setlang en` — it is not specific to non-English languages.

8.4 The two standard workflows

Workflow A — language first, then speaker

```
python setup_lang.py setlang de          # guard: (re)installs jd3, installs
the de section
python install_speaker.py install s2     # switch to s2; the de section travels
with the swap
vtl-synth run "ich suche eine schoene Strasse" --lang de --expressive -o ./out
```

Result: expressive German on s2. The F0 contour follows s2's `f0_default`; the breath groups and accents are identical to jd3's. The vowel timbre is the one caveat — see below.

Workflow B — speaker first

```
python install_speaker.py install s2
python setup_lang.py setlang it          # guard reinstalls jd3 (unavoidable by
design)
python install_speaker.py install s2     # re-switch; the it section travels
vtl-synth run "la famiglia vive in campagna" --lang it --expressive -o ./out
```

The double switch is the price of the guard's simplicity; users who alternate languages and speakers should adopt workflow A.

8.5 Guarantees matrix

	jd3 active	other speaker active
Expressive synthesis, any language	fully validated (measurements, demos, tests)	works; F0 relief identical in semitones (validated on s2 by smoke test + measurements planned)
Per-language vowel quality	reference (calibrated in situ)	hypothesis: JD3-calibrated targets applied to another geometry; flagged by <code>setup_lang.py show</code>
calibrate / verify for the speaker	available	deliberately unavailable (guard reinstalls jd3)
Nuclear fall / accent amplitude	per-language profile, speaker-independent	same profile, scaled to the speaker's <code>f0_default</code>

One-sentence summary

The guard keeps *language calibration* anchored to JD3; expressive prosody follows *whichever speaker is active*. The two mechanisms coexist without interacting, and the option never modifies the speaker setup.

9 Limitations (documented heuristics)

- The chunker is lexical: rare conjunction senses (*that* as complementiser vs. determiner is handled, but e.g. *like* as preposition is not in the list), and the balanced hard-limit cut can fall at an unremarkable spot.
- German stress = first syllable: prefixes (*ver-*, *be-*, *ge-*) are not modelled.
- The shipped Wikipron TSVs carry no stress marks; the stress-mark-capture path activates with a narrow lexicon.
- De-accentuation is a first/last-lexical-word heuristic (a coarse given/new stand-in), not discourse modelling (Pierrehumbert’s H*+deaccentuation is cited as the target, not implemented).
- The contour is a 2–3-parameter caricature of the cited ToBI systems (one target shape per language), chosen to be tunable rather than exhaustive.

10 File map

file	role
<code>vtl_synth/utils/chunking.py</code>	breath-group chunking, per-language word lists
<code>vtl_synth/utils/prosody_f0.py</code>	ExpressivityProfile, stress channel consumption, <code>apply_expressive_f0</code>
<code>vtl_synth/core/syltraj.py</code>	% token, T_pause_breath, <code>get_last_build()</code>
<code>vtl_synth/core/build_phrase_tract.py</code>	<code>pause_breath_ms</code> passthrough
<code>vtl_synth/core/pipeline.py</code>	<code>Pipeline(expressive=...)</code> orchestration
<code>vtl_synth/cli/main.py</code>	<code>--expressive / --no-expressive</code> , banner
<code>vtl_synth/utils/lexicon_loader.py</code>	<code>ipa_to_keys_stress</code> , <code>stress_for</code>
<code>vtl_synth/utils/setlang.py</code>	<code>LangProfile.expressivity</code>
<code>scripts/f0_report.py</code>	F0 measurements (accents, smoothness, span)
<code>examples/demo_prosody.py</code>	before/after demo, six languages
<code>tests/test_expressive.py</code>	40-test battery incl. bit-identity
<code>lang_pack/profiles/{chunking,prosody_f0}.py</code>	shipped copies (single source of truth)